Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 1

Синхронные одноступенчатые триггеры со статическим и динамическим управлением записью

Студент Михайлов А.К.

Группа ФН12-41Б

Преподаватель Попов А.Ю.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Инфиксная нотация 5

1.2 Постфиксная нотация 5

1.3 Стек 5

2 Конструкторская часть **6**

2.1 Структура Stack 6

2.2 Алгоритм токенизации строки 6

2.3 Алгоритм преобразования инфиксной записи в постфиксную 6

2.4 Алгоритм вычисления постфиксного выражения 8

3 Технологическая часть **12**

3.1 Реализация и выбор средств 12

3.2 Тестирование программы 12

ЗАКЛЮЧЕНИЕ **14**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **15**

ПРИЛОЖЕНИЕ А **16**

ВВЕДЕНИЕ

В лабораторной работе рассматривается алгоритм преобразования **инфиксных** арифметических выражений в **обратную польскую запись**, а также вычисление с помощью неё значений.

Целью работы является описание и реализация алгоритма преобразования инфиксной записи в постфиксную с последующим вычислением в **обратной польской записи** (ОПЗ).

Для достижения поставленной задачи необходимо выполнить следующие пункты:

1) описать инфиксную и постфиксную записи, а также дать определения основных понятий, требуемых для преобразования из первой во вторую;
2) разработать алгоритмы, позволяющие преобразовывать инфиксную запись в постфиксную и после проводить в ней вычисления;
3) реализовать описанные алгоритмы;
4) провести тестирование реализации разработанных алгоритмов.

Задание индивидуального варианта: даны неотрицательные целые числа в двоичной системе счисления, максимум 8 разрядов, операции над ними — все виды сравнений, двоичные сдвиги, побитовое отрицание.

1 Аналитическая часть

1.1 Инфиксная нотация

Инфиксная нотация — это способ записи математических выражений, в котором операторы (например, $+$ и $\cdot$) располагаются между операндами. Этот формат используется в математике и привычен для человека, так как легко читается. Однако для обработки компьютерами инфиксная нотация не используется, поскольку требует учёта порядка операций (приоритетов) и наличия скобок для уточнения последовательности вычислений.

1.2 Постфиксная нотация

Постфиксная нотация (или **обратная польская нотация**) — это способ записи математических выражений, в котором операторы записываются после операндов. Такой формат удобен для вычислений компьютерами и упрощает обработку, поскольку устраняет необходимость в скобках и учёте приоритетов операций. Операции выполняются строго слева направо в порядке появления оператора, что делает её эффективной для реализации в стековых вычислительных алгоритмах.

1.3 Стек

Стек — это структура данных, которая позволяет извлечь только последний помещённый в неё элемент (то есть работает по системе $LIFO$, "последний зашёл, первый вышел"(см. рисунок 1.1)) У стека есть две основные операции `push` и `pop` — добавление элемента в стек извлечение из него элемента соответственно.

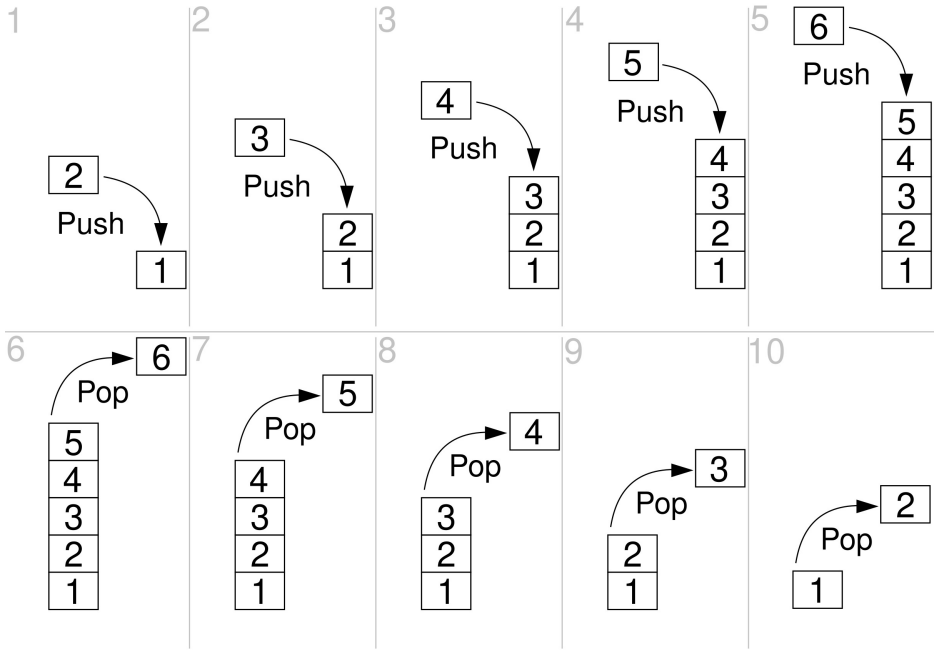

Рисунок 1.1 — Схема работы стека

2 Конструкторская часть

В данном разделе представлены формальные описания алгоритмов преобразования входной строки в последовательность токенов, трансляции выражений из инфиксной записи в постфиксную, обработки и расчёта значений выражений, записанных в постфиксной форме. Все алгоритмы реализованы в соответствии с требованиями варианта. Для написания псевдокода использовался пакет `algorithmicx` для `latex` [2].

2.1 Структура Stack

Структура **Stack** реализует классический стек с основными операциями:

— `push` — добавление элемента на вершину стека;

— `pop` — удаление и возврат элемента с вершины стека (выбрасывает исключение, если стек пуст);

— `peek` — просмотр элемента на вершине стека без его удаления (выбрасывает исключение, если стек пуст);

— `isEmpty` — проверка, пуст ли стек.

Для хранения данных используется контейнер `data`.

2.2 Алгоритм токенизации строки

Функция `tokenize` выполняет лексический анализ строки и преобразует её в последовательность токенов. На вход подается строка, содержащая математическое выражени. Возвращается вектор строк, содержащий последовательность токенов. Токены включают десятичные и двоичные операнды, а также операторы сравнения ($>$, $<$, $>=$, $<=$, $=$, $!=$), операторы двоичных сдвигов ($<<$, $>>$) и оператор побитового отрицания ($\sim$).

2.3 Алгоритм преобразования инфиксной записи в постфиксную

Функция `infixToPostfix` преобразует последовательность токенов инфиксного выражения в постфиксную запись. На вход подается последовательность токенов, возвращается вектор постфиксной записи. Также были реализованы вспомогательные функции `isBinaryNumber`, `isDecimalNumber` и `isOperator`, которые проверяют, является ли токен двоичным или десятичным числом или оператором соответственно, а также фукнция `getPrecedence`, определяющая приоритет операторов.

В таблице 2.1 показаны приоритеты операторов, где 4 — самый высокий приоритет (выполняется в первую очередь).

Таблица 2.1 — Приоритеты операторов

Оператор	**Приоритет**
$\sim$	4
$\ll$, $\gg$	3
<, >, <=, >=	2
==, ! =	1

На рисунках 2.1 — 2.2 представлено описание алгоритма преобразования инфиксной записи в постфиксную с помощью псевдокода.

```
procedure infixToPostfix(tokens)
    output ← пустой вектор строк
    operatorStack ← пустой стек строк
    for каждого token в tokens do
        if isBinaryNumber(token) или isDecimalNumber(token) then
            output.push_back(token)
        else if isUnaryOperator(token) then
            operatorStack.push(token)
        else if isOperator(token) then
            while не operatorStack.isEmpty() и operatorStack.peek() ≠ "(" и
getPrecedence(operatorStack.peek()) ≥ getPrecedence(token) do
                output.push_back(operatorStack.pop())
            end while
            operatorStack.push(token)
        else if token = "(" then
            operatorStack.push(token)
        else if token = ")" then
            while не operatorStack.isEmpty() и operatorStack.peek() ≠ "(" do
                output.push_back(operatorStack.pop())
            end while
            if operatorStack.isEmpty() then
                throw "Несогласованные скобки (код -1)"
            end if
            operatorStack.pop()
            if не operatorStack.isEmpty() и isUnaryOperator(operatorStack.peek()) then
                output.push_back(operatorStack.pop())
            end if
```

Рисунок 2.1 — Алгоритм преобразования инфиксной записи в постфиксную, часть 1

```
        else
            throw “Неизвестный токен: ” + token + “ (код -1) ”
        end if
    end for
    while не operatorStack.isEmpty() do
        if operatorStack.peek() = “(” или operatorStack.peek() = “)” then
            throw “Несогласованные скобки (код -1)”
        end if
        output.push_back(operatorStack.pop())
    end while
    return output
end procedure
```

Рисунок 2.2 — Алгоритм преобразования инфиксной записи в постфиксную, часть 2

2.4 Алгоритм вычисления постфиксного выражения

Функция `evaluatePostfix` выполняет вычисление выражения в постфиксной записи. На вход подается вектор постфиксной записи, возвращается строка — результат вычислений. Также были реализованы вспомогательные функции `binaryToDecimal` и `decimalToBinary`, которые переводят число из десятичной системы счисления в двоичную и наоборот. На рисунках 2.3 — 2.6 представлено описание алгоритма вычисления постфиксного выражения с помощью псевдокода.

```
procedure evaluatePostfix(postfix)
    stack ← пустой стек строк
    for каждого token в postfix do
        if isBinaryNumber(token) или isDecimalNumber(token) then
            stack.push(token)
        else if isOperator(token) then
            if isUnaryOperator(token) then
                if stack.isEmpty() then
                    throw “Недостаточно операндов для оператора ∼ (код -2)”
                end if
                operand ← stack.pop()
                if не isBinaryNumber(operand) then
                    throw “Оператор ∼ применяется только к двоичным числам: ” + operand
+ “ (код -2)”
                end if
                result ← пустая строка
```

Рисунок 2.3 — Алгоритм вычисления постфиксного выражения, часть 1

```
16:                 for каждого c в operand do
17:                     if c = '0' then
18:                         result ← result + '1'
19:                     else
20:                         result ← result + '0'
21:                     end if
22:                 end for
23:                 stack.push(result)
24:             else
25:                 if stack.isEmpty() then
26:                     throw "Недостаточно операндов для бинарного оператора (код -2)"
27:                 end if
28:                 right ← stack.pop()
29:                 if stack.isEmpty() then
30:                     throw "Недостаточно операндов для бинарного оператора (код -2)"
31:                 end if
32:                 left ← stack.pop()
33:                 if token = "«" или token = "»" then
34:                     if не isBinaryNumber(left) then
35:                         throw "Левый операнд оператора " + token + " должен быть двоичным
    числом: " + left + " (код -2)"
36:                     end if
37:                     if не isDecimalNumber(right) then
38:                         throw "Правый операнд оператора " + token + " должен быть десятич-
    ным числом: " + right + " (код -2)"
39:                     end if
40:                     shiftAmount ← stoi(right)
41:                     if shiftAmount < 0 then
42:                         throw "Количество битов для сдвига не может быть отрицательным: "
    + to_string(shiftAmount) + " (код -2)"
43:                     else if shiftAmount > 8 then
44:                         throw "Слишком большое количество битов для сдвига: " +
    to_string(shiftAmount) + " (код -2)"
45:                     end if
46:                     result ← left
47:                     if token = "«" then
48:                         for i ← 0 до shiftAmount − 1 do
49:                             result ← result + '0'
50:                         end for
51:                     else
52:                         if shiftAmount ≥ result.length() then
53:                             result ← '0'
54:                         else
55:                             result ← result.substr(0, result.length() − shiftAmount)
56:                         end if
57:                     end if
58:                     stack.push(result)
```

Рисунок 2.4 — Алгоритм вычисления постфиксного выражения, часть 2

```
        else
            leftDec, rightDec ← 0
            if isBinaryNumber(left) then
                leftDec ← binaryToDecimal(left)
            else if isDecimalNumber(left) then
                leftDec ← stoi(left)
            else
                throw “Неизвестный формат числа: ” + left + “ (код -1)”
            end if
            if isBinaryNumber(right) then
                rightDec ← binaryToDecimal(right)
            else if isDecimalNumber(right) then
                rightDec ← stoi(right)
            else
                throw “Неизвестный формат числа: ” + right + “ (код -1)”
            end if
            result ← пустая строка
            if token = “=” then
                if leftDec = rightDec then
                    result ← ‘1’
                else
                    result ← ‘0’
                end if
            else if token = “!=” then
                if leftDec ≠ rightDec then
                    result ← ‘1’
                else
                    result ← ‘0’
                end if
            else if token = “<” then
                if leftDec < rightDec then
                    result ← ‘1’
                else
                    result ← ‘0’
                end if
            else if token = “>” then
                if leftDec > rightDec then
                    result ← ‘1’
                else
                    result ← ‘0’
                end if
            else if token = “<=” then
                if leftDec ≤ rightDec then
                    result ← ‘1’
                else
                    result ← ‘0’
                end if
```

Рисунок 2.5 — Алгоритм вычисления постфиксного выражения, часть 3

```
                else if token = “>=” then
                    if leftDec ≥ rightDec then
                        result ← ‘1’
                    else
                        result ← ‘0’
                    end if
                else
                    throw “Неизвестный оператор: ” + token + “ (код -1)”
                end if
                stack.push(result)
            end if
        end if
    else
        throw “Неизвестный токен при вычислении: ” + token + “ (код -1)”
    end if
end for
if stack.isEmpty() then
    throw “Результат не найден”
end if
result ← stack.pop()
if не stack.isEmpty() then
    throw “Лишние операнды в выражении (код -2)”
end if
return result
end procedure
```

Рисунок 2.6 — Алгоритм вычисления постфиксного выражения, часть 4

Вывод

Была разработана структура **Stack**, обеспечивающая базовые операции работы со стеком, а также алгоритмы токенизации, преобразования инфиксной записи в постфиксную и вычисления выражений с поддержкой операций над двоичными числами.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации использовалась среда разработки *Visual Studio Code* [3]. Язык программирования, на котором была выполнена реализация алгоритмов — $C++$. Реализация программы содержит структуру `Stack`, обеспечивающую базовые операции работы со стеком, а также методы, описанные в предыдущем разделе. В листинге А.1 представлена программная реализация описанных алгоритмов и структуры.

3.2 Тестирование программы

Если во время работы программы возникают ошибки, она завершает свою работу, выводя код и описание ошибки. Коды и описания ошибок, которые могут произойти во время исполнения:

1) -1 — невозможно перевести выражение из инфиксной в постфиксную форму записи из за неправильной расстановки скобок или некорректных имен операторов или операндов;
2) -2 — невозможно вычислить значение выражения из за недостаточного количества операндов для данного оператора.

В таблице 3.1 приведены результаты тестирования программы.

Таблица 3.1 — Результаты тестирования

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на работу с некорректными данными	A	ОШИБКА: Неизвестный токен: A (код -1)	ОШИБКА: Неизвестный токен: A (код -1)
Проверка работы с операциями двоичного сдвига	$1001 \ll 2$	100100	100100
	$1100 \ll 1$	110	110
	$2 \ll 1$	ОШИБКА: Левый операнд оператора « должен быть двоичным числом (код -2)	ОШИБКА: Левый операнд оператора « должен быть двоичным числом (код -2)
	$101 \ll 10$	ОШИБКА: Слишком большое количество битов для сдвига (код -2)	ОШИБКА: Слишком большое количество битов для сдвига (код -2)

Таблица 3.1 — Продолжение

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
	1001 ≪ -1	ОШИБКА: Количество битов для сдвига не может быть отрицательным (код -2)	ОШИБКА: Количество битов для сдвига не может быть отрицательным (код -2)
Проверка работы с унарным минусом	∼ 1010	0101	0101
	∼ 5	ОШИБКА: Оператор ∼ применяется только к двоичным числам (код -2)	ОШИБКА: Оператор ∼ применяется только к двоичным числам (код -2)
Проверка работы с операциями сравнения	10 = 10	1	1
	10 != 101	1	1
	101 > 10	1	1
	101 < 10	0	0
	101 <= 10	0	0
	10 >= 10	1	1
	21 = 10	ОШИБКА: Нельзя сравнивать десятичные числа (код -2)	ОШИБКА: Нельзя сравнивать десятичные числа (код -2)
Проверка выражений со скобками	(1010 ≪ 1) = 10100	1	1
Проверка на недостаточное количество операндов	1010 ≪	ОШИБКА: Недостаточно операндов для бинарного оператора (код -2)	ОШИБКА: Недостаточно операндов для бинарного оператора (код -2)
Проверка несогласованных скобок	(1010 ≪ 1	ОШИБКА: Несогласованные скобки (код -1)	ОШИБКА: Несогласованные скобки (код -1)

Вывод

Программа успешно прошла тестирование на работоспособность.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы, а именно реализованы алгоритмы преобразования инфиксной записи в постфиксную с последующим вычислением в обратной польской записи.

Также были выполнены все задачи:

1) описаны инфиксная и постфиксная записи, а также даны определения основных понятий, требуемых для преобразования из первой во вторую;
2) разработаны алгоритмы, позволяющие преобразовывать инфиксную запись в постфиксную и после проводить в ней вычисления;
3) реализованы описанные алгоритмы;
4) проведено тестирование реализации разработанных алгоритмов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы современных алгоритмов 2-е дополненное издание / Дж. Макконелл // под ред. С.Кландо, доп. М.В.Ульянова. — Москва: ТЕХНОСФЕРА, 2004. — 370 с.

2. Псевдокод алгоритма в TeX [Электронный ресурс]. URL: https://pro-prof.com/forums/topic/псевдокод-алгоритма-в-tex (дата обращения: 20 ноября 2025).

3. Visual Studio Code documentation. [Электронный ресурс]. URL: https://code.visualstudio.com/docs (дата обращения: 19 ноября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```
##include <iostream>
#include <vector>
#include <string>
#include <stdexcept>
#include <unordered_map>
#include <cmath>

using namespace std;

template <typename T>
class Stack {
  private:
  vector<T> data;
  public:
  void push(T value) { data.push_back(value); }
  T pop() {
    if (isEmpty())
    throw runtime_error("Стек пустой");
    T value = data.back();
    data.pop_back();
    return value;
  }
  T peek() const {
    if (isEmpty())
    throw runtime_error("Стек пустой");
    return data.back();
  }
  bool isEmpty() const {
    return data.empty();
  }
};

bool isBinaryNumber(const string& str) {
  if (str.empty()) return false;
  for (char c : str) {
    if (c != '0' && c != '1') return false;
  }
  return str.length() <= 8;
}

bool isDecimalNumber(const string& str) {
  if (str.empty())
  return false;
  for (char c : str)
```

```
  if (!isdigit(c))
  return false;
  return true;
}

int getPrecedence(const string& op) {
  if (op == "~") return 4;
  if (op == "<<" || op == ">>") return 3;
  if (op == "<" || op == ">" || op == "<=" || op == ">=") return 2;
  if (op == "=" || op == "!=") return 1;
  return 0;
}

bool isOperator(const string& token) {
  return token == "<<" || token == ">>" || token == "~" ||
  token == "=" || token == "!=" || token == "<" ||
  token == ">" || token == "<=" || token == ">=";
}

bool isUnaryOperator(const string& token) {
  return token == "~";
}

int binaryToDecimal(const string& binary) {
  int result = 0;
  for (char c : binary)
  result = result * 2 + (c - '0');
  return result;
}

string decimalToBinary(int num) {
  if (num == 0) return "0";
  string result;
  while (num > 0) {
    result = char('0' + (num % 2)) + result;
    num /= 2;
  }
  return result;
}

string evaluatePostfix(const vector<string>& postfix) {
  Stack<string> stack;

  for (const auto& token : postfix) {
    if (isBinaryNumber(token) || isDecimalNumber(token))
    stack.push(token);
    else if (isOperator(token)) {
```

```
    if (isUnaryOperator(token)) {
      if (stack.isEmpty())
      throw runtime_error("Недостаточно операндов для оператора ~ код( -2)");
      string operand = stack.pop();

      if (!isBinaryNumber(operand))
      throw runtime_error("Оператор ~ применяется только к двоичным числам: код( -2)" + operand);

      string result;
      for (char c : operand)
      result += (c == '0') ? '1' : '0';
      stack.push(result);
    }
    else {
      if (stack.isEmpty())
      throw runtime_error("Недостаточно операндов для бинарного оператора код( -2)");
      string right = stack.pop();
      if (stack.isEmpty())
      throw runtime_error("Недостаточно операндов для бинарного оператора код( -2)");
      string left = stack.pop();

      if (token == "<<" || token == ">>") {
        if (!isBinaryNumber(left))
        throw runtime_error("Левый операнд оператора " + token + " должен быть двоичным числом: код( -2)" + left);
        if (!isDecimalNumber(right))
        throw runtime_error("Правый операнд оператора " + token + " должен быть десятичным числом: код( -2)" + right);

        int shiftAmount = stoi(right);

        if (shiftAmount < 0)
        throw runtime_error("Количество битов для сдвига не может быть отрицательным: код( -2)" + to_string(shiftAmount));

        else if (shiftAmount > 8)
        throw runtime_error("Слишком большое количество битов для сдвига: " + to_string(shiftAmount) + " код( -2)");

        string result = left;

        if (token == "<<") {
          for (int i = 0; i < shiftAmount; i++)
          result += '0';
```

```
        }
        else {
          if (shiftAmount >= result.length())
          result = "0";
          else
          result = result.substr(0, result.length() - shiftAmount);
        }

        stack.push(result);
      }
      else {
        if (isDecimalNumber(left) && !isBinaryNumber(left) ||
isDecimalNumber(right) && !isBinaryNumber(right))
        throw runtime_error("Нельзя сравнивать десятичные числа: " + left + " и
" + right + " код( -2)");

        int leftDec, rightDec;

        if (isBinaryNumber(left))
        leftDec = binaryToDecimal(left);
        else if (isDecimalNumber(left))
        leftDec = stoi(left);
        else
        throw runtime_error("Неизвестный формат числа: " + left + " код( -1)")
;

        if (isBinaryNumber(right))
        rightDec = binaryToDecimal(right);
        else if (isDecimalNumber(right))
        rightDec = stoi(right);
        else
        throw runtime_error("Неизвестный формат числа: " + right + " код( -1)"
);

        string result;
        if (token == "=")
        result = (leftDec == rightDec) ? "1" : "0";

        else if (token == "!=")
        result = (leftDec != rightDec) ? "1" : "0";

        else if (token == "<")
        result = (leftDec < rightDec) ? "1" : "0";

        else if (token == ">")
        result = (leftDec > rightDec) ? "1" : "0";

```

```
          else if (token == "<=")
          result = (leftDec <= rightDec) ? "1" : "0";

          else if (token == ">=")
          result = (leftDec >= rightDec) ? "1" : "0";

          else
          throw runtime_error("Неизвестный оператор: " + token + " код( -1)");

          stack.push(result);
        }
      }
    }
    else
    throw runtime_error("Неизвестный токен при вычислении: " + token + " код( -1)"
  );
  }

  if (stack.isEmpty())
  throw runtime_error("Результат не найден");
  string result = stack.pop();
  if (!stack.isEmpty())
  throw runtime_error("Лишние операнды в выражении код( -1)");

  return result;
}

vector<string> infixToPostfix(const vector<string>& tokens) {
  vector<string> output;
  Stack<string> operatorStack;

  for (const auto& token : tokens) {
    if (isBinaryNumber(token) || isDecimalNumber(token))
    output.push_back(token);

    else if (isUnaryOperator(token))
    operatorStack.push(token);

    else if (isOperator(token)) {
      while (!operatorStack.isEmpty() &&
      operatorStack.peek() != "(" &&
      getPrecedence(operatorStack.peek()) >= getPrecedence(token)) {
        output.push_back(operatorStack.pop());
      }
      operatorStack.push(token);
    }

```

```
    else if (token == "(")
    operatorStack.push(token);

    else if (token == ")") {
      while (!operatorStack.isEmpty() && operatorStack.peek() != "(")
      output.push_back(operatorStack.pop());

      if (operatorStack.isEmpty())
      throw runtime_error("Несогласованные скобки код( -1)");
      operatorStack.pop();

      if (!operatorStack.isEmpty() && isUnaryOperator(operatorStack.peek()))
      output.push_back(operatorStack.pop());
    }
    else
    throw runtime_error("Неизвестный токен: " + token + " код( -1)");
  }

  while (!operatorStack.isEmpty()) {
    if (operatorStack.peek() == "(" || operatorStack.peek() == ")")
    throw runtime_error("Несогласованные скобки код( -1)");
    output.push_back(operatorStack.pop());
  }

  return output;
}

vector<string> tokenize(const string& expr) {
  vector<string> tokens;
  string currentToken;

  for (int i = 0; i < expr.length(); ++i) {
    char c = expr[i];

    if (isspace(c)) {
      if (!currentToken.empty()) {
        tokens.push_back(currentToken);
        currentToken.clear();
      }
      continue;
    }

    if (c == '0' || c == '1') {
      if (!currentToken.empty() && !(currentToken[0] == '0' || currentToken
    [0] == '1')) {
        tokens.push_back(currentToken);
        currentToken.clear();
```

```
      }

      currentToken += c;

      for (size_t j = i + 1; j < expr.length() && j < i + 8; ++j) {
        if (expr[j] == '0' || expr[j] == '1') {
          currentToken += expr[j];
          i = j;
        }
        else
        break;
      }

      tokens.push_back(currentToken);
      currentToken.clear();
    }

    else if (c == '=' || c == '!' || c == '<' || c == '>') {
      if (!currentToken.empty()) {
        tokens.push_back(currentToken);
        currentToken.clear();
      }

      currentToken += c;

      if (i + 1 < expr.length()) {
        char next = expr[i + 1];
        if ((c == '!' && next == '=') ||
        (c == '<' && next == '=') ||
        (c == '>' && next == '=') ||
        (c == '<' && next == '<') ||
        (c == '>' && next == '>')) {
          currentToken += next;
          i++;
        }
      }

      tokens.push_back(currentToken);
      currentToken.clear();
    }

    else if (c == '~') {
      if (!currentToken.empty()) {
        tokens.push_back(currentToken);
        currentToken.clear();
      }

```

```
      tokens.push_back(string(1, c));
    }
    else if (isdigit(c)) {
      if (!currentToken.empty() && !isdigit(currentToken[0])) {
        tokens.push_back(currentToken);
        currentToken.clear();
      }

      currentToken += c;

      while (i + 1 < expr.length() && isdigit(expr[i + 1]))
      currentToken += expr[++i];

      tokens.push_back(currentToken);
      currentToken.clear();
    }

    else {
      if (!currentToken.empty()) {
        tokens.push_back(currentToken);
        currentToken.clear();
      }
      tokens.push_back(string(1, c));
    }
  }

  if (!currentToken.empty())
  tokens.push_back(currentToken);

  return tokens;
}

int getErrorCode(const string& errorMessage) {
  if (errorMessage.find("код( -1)") != string::npos)
  return -1;
  else if (errorMessage.find("код( -2)") != string::npos)
  return -2;
  return -1;
}

int main() {
  cout << "<<< Калькулятор бинарных выражений >>>" << endl;
  cout << "Поддерживаемые операторы: << >> ~ == != < > <= >=" << endl;
  cout << "Десятичные числа и двоичные числа до 8 бит" << endl;
  cout << "Для выхода введите 'exit' или 'quit'" << endl;

  string userInput;
```

```

  while (true) {
    cout << "\Введитеn выражение: ";
    getline(cin, userInput);

    if (userInput == "exit" || userInput == "quit" || userInput == "выход") {
      cout << "До свидания!" << endl;
      break;
    }

    if (userInput.empty())
    continue;

    try {
      vector<string> tokens = tokenize(userInput);

      vector<string> postfix = infixToPostfix(tokens);

      cout << "Постфиксная запись: ";
      for (const auto& token : postfix)
      cout << token << " ";
      cout << endl;

      string result = evaluatePostfix(postfix);
      cout << "Результат: " << result << endl;

    } catch (const exception& e) {
      string errorMessage = e.what();
      cout << "ОШИБКА: " << errorMessage << endl;

      int errorCode = getErrorCode(errorMessage);
      return errorCode;
    }
  }

  return 0;
}
```

Листинг А.1 — Программная реализация описанных алгоритмов